

Operating Systems (CS-2006)

Date: 9/04/2026

Course Instructors

Dr. Ghufuran Ahmed, Ms. Anaum Hamid

Mr. Minhaj Raza, Mr. Faisal Ali, Ms. Saeeda,

Mr. Zulfiqar Ali, Ms. Rabia, Mr. Ubaidullah

Sessional-II Exam

Total Time: 1 hour

Total Marks: 15

Total Questions: 03

Semester: Spring 2026

Campus: Karachi

Dept. CS, SE, DS&AI, CY

Roll No

Section

Student Signature

Do not use markers or pencils to write your answers

CLO # 3: Design and Implement programs using processes and threads.

Question 1: (Est. Time: 5 + 5 + 10 = 20 Minutes) (1.5 + 1.5 + 3 = 6 points)

- a) Modern CPUs often reorder instructions to save time—how does this "efficiency" break classic synchronization logic?

Solution:

Modern CPUs reorder instructions to run faster, but that breaks classic synchronization logic.

Three problems pop up:

Instructions don't always run in the order you wrote them — so your code logic looks correct on paper but fails at runtime.

Classic algorithms like Peterson's solution assume strict step-by-step execution. Modern CPUs don't guarantee that anymore, so two threads can end up in the critical section together even when the algorithm says it shouldn't happen.

So you need memory barriers or atomic operations with ordering guarantees.

- b) A programmer spends \$5,000 on a fancy new 32-core server to speed up a data processing application. After running the code, they are shocked to find that the program is only 2 times faster than it was on their old 2-core laptop. Your Task is to explain the logical reason why adding 30 more cores didn't result in a 30x speedup. What part of the program's "logic" (serial vs. parallel) is likely holding the performance back?

Solution:

The performance is limited by Amdahl's Law.

Logical reasoning.

Even with 32 cores, if a fraction s of the program is inherently serial (e.g., sequential loops, shared resource contention, I/O, or synchronization bottlenecks), the theoretical maximum speedup is:

$$\text{Speedup} \leq \frac{1}{s + \frac{1-s}{n}}$$

where n is the number of cores.

- If only 50% of the program is parallelizable, even with infinite cores, speedup cannot exceed 2x.

Following things force serial behavior?

- Plain old sequential code
- Locks (mutexes, spinlocks) that make threads wait
- Overhead from keeping caches in sync
- Contention over shared memory
- I/O or memory bandwidth hitting a wall

c) A company is designing a web server. They have two design proposals:

Design 1 (Thread-per-Request): Create a new thread for every incoming HTTP request. The thread handles the request and then terminates.

Design 2 (Process-per-Request with Fork-Join): A master process accepts incoming requests. For each request, the master process calls `fork()` to create a child process. The child process handles the request and then terminates. The master process uses `wait()` or `waitpid()` to reap terminated child processes.

Questions:

- Compare the resource usage (memory, CPU, and creation overhead) between the two designs under normal load.
- Which design is more robust under a "Denial of Service (DoS) attack" where an attacker sends thousands of fake requests per second? Justify your answer.

Solution:

Resource Usage

Design 1: Thread-per-Request

This design keeps memory usage low because all threads share the same address space — you only need a small stack for each thread. It's also pretty light on the CPU since there's less overhead when switching between threads. And creating a thread is fast; it's a lightweight operation.

Design 2: Process-per-Request (using fork)

This one takes more memory. Every process gets its own separate address space, and even with copy-on-write optimizations, it still adds up. CPU usage is higher too, because switching between processes is heavier than switching between threads. Plus, forking a new process is slow compared to creating a thread.

Robustness under DoS Attack

Thread-per-request is more robust under DoS because threads are lightweight, allowing the server to handle a large number of requests with lower memory and creation overhead.

Process-per-request quickly exhausts system resources (memory, process table) under thousands of requests due to heavy `fork()` cost. Although processes provide better isolation, they scale poorly under high-load attacks. Hence, threads are preferred for better scalability under DoS conditions.

CLO # 2: Implementation of concurrency and synchronization mechanism

Question 2: (Est. Time: 10 + 20 + 10 = 40 Minutes) (2 + 5 + 2 = 9 points)

- a) Identify & correct the potential problem in the following multithread pseudo-code for a bank account system. Two threads, DepositThread & WithdrawThread, run concurrently and share the variable balance.

```
// Shared Data:
int balance = 100;
void deposit(int amount) {
    int temp = balance;    // Read balance
    temp = temp + amount;  // Add amount
    balance = temp;        // Write back balance }
void withdraw(int amount) {
    int temp = balance;    // Read balance
    temp = temp - amount;  // Subtract amount
    balance = temp;        // Write back balance
}
```

Solution:

The Problem here is Race Condition

Both deposit() and withdraw() do the classic read-modify-write pattern:

Read balance into temp

Update temp (add or subtract)

Write temp back to balance

Here is the explanation

if deposit(50) and withdraw(30) run at the same time:

- Thread A (deposit) reads balance = 100 → temp = 100
- Thread B (withdraw) reads balance = 100 (still 100 because A hasn't written back yet) → temp = 100
- Thread A does temp = 100 + 50 = 150
- Thread B does temp = 100 - 30 = 70
- Thread A writes balance = 150
- Thread B writes balance = 70

Final balance: 70. But it should be 100 + 50 - 30 = 120.

Correction Solution

```
// Shared Data:
int balance = 100;
mutex_t balance_mutex;

void deposit(int amount) {
    mutex_lock(&balance_mutex);
    int temp = balance;
    temp = temp + amount;
    balance = temp;
    mutex_unlock(&balance_mutex);
}
```

```
void withdraw(int amount) {
    mutex_lock(&balance_mutex);
    int temp = balance;
    temp = temp - amount;
    balance = temp;
    mutex_unlock(&balance_mutex);
}
```

allows us

can perform read and write operations simultaneously. Occasionally, an administrator updates or edits

the books. The current system is designed such that: If at least one student/teacher is reading, new students are allowed to join immediately Even if an administrator is waiting, students/teacher are given priority. Over time, administrators complain that they are unable to update books because there are always students/teachers accessing them. Your Tasks:

- i. Identify above scenario matches to which Classical synchronization problem
- ii. Explain briefly why this system leads to administrator in starvation.
- iii. Propose a modification to make the system fair
- iv. Write the Code for the improved solution.

Solution:

- i) This scenario matches the Readers-Writers Problem,
- ii) In the current system, administrators experience starvation because:(As per Reader Writer Problem)Readers preference allows unlimited readers to join even when writers are waiting. The condition "if at least one student/teacher is reading, new students are allowed to join immediately" means writers never get a chance to acquire the resource Writers can be perpetually postponed if readers continuously arrive, leading to starvation
- iii) Implement Writers Preference or Fair Priority solution with these changes:

- Count waiting writers and give them priority
 - Once a writer is waiting, no new readers can start
 - After writer completes, allow waiting readers to proceed
- Use semaphores/mutexes to control access and track counts

iv) Code

```
// Shared data
int readers = 0;
int writer_waiting = 0; // Track if a writer is waiting
mutex_t m;
cond_t cond;
void reader() {
    mutex_lock(&m);
    while (writer_waiting > 0) // Block if a writer is
        waiting
        cond_wait(&cond, &m);
    readers++;
    mutex_unlock(&m);

    // Read
    mutex_lock(&m);
    readers--;
    if (readers == 0)
        cond_signal(&cond); // Signal waiting writer
    mutex_unlock(&m);
}
```

```
void writer() {
    mutex_lock(&m);
    writer_waiting = 1; // Announce arrival
    while (readers > 0) // Wait for active readers
        to finish
        cond_wait(&cond, &m);
    mutex_unlock(&m);

    // Write

    mutex_lock(&m);
    writer_waiting = 0; // Done writing
    cond_broadcast(&cond); // Wake all waiting
    readers
    mutex_unlock(&m);
}
```

- c) A system has 256 MB of RAM. The average process size is 20 MB, and the OS itself occupies 16 MB. The system administrator can configure the degree of multiprogramming (number of processes in memory). However, as more processes are loaded, I/O wait time increases. The CPU utilization can be modeled as: $\text{CPU Utilization} = 1 - (\text{I/O Wait})^n$ where n is the degree of multiprogramming, and I/O wait for a single process is 80%.
- I. What is the maximum number of processes that can fit in memory using contiguous allocation without considering fragmentation?
 - II. Calculate CPU utilization for $n = 1, 2, 3$, and 4.

I. Maximum Number of Processes (Contiguous Allocation)

Given:

- Total RAM = 256 MB
- OS overhead = 16 MB
- Available for processes = $256 - 16 = \mathbf{240 \text{ MB}}$
- Average process size = 20 MB

Calculation:

$$\text{Max processes} = \frac{240 \text{ MB}}{20 \text{ MB}} = 12$$

Answer: Maximum **12 processes** can fit in memory.

. CPU Utilization for $n = 1, 2, 3, 4$

Given:

- I/O wait for a single process = 80% = 0.8
- Formula: $\text{CPU Utilization} = 1 - (0.8)^n$

n	Calculation	CPU Utilization
1	$1 - (0.8)^1 = 1 - 0.8$	0.20 (20%)
2	$1 - (0.8)^2 = 1 - 0.64$	0.36 (36%)
3	$1 - (0.8)^3 = 1 - 0.512$	0.488 (48.8%)
4	$1 - (0.8)^4 = 1 - 0.4096$	0.5904 (59.04%)